

AKSARAY ÜNİVERSİTESİ

İKTİSADİ VE İDARİ BİLİMLER FAKÜLTESİ

YÖNETİM BİLİŞİM SİSTEMLERİ BÖLÜMÜ

DERS: Veri Tabanı Yönetim Sistemleri (VTYS)

KONU: SQL Programlama ve Veritabanı Tasarımı Ders Notları

DERS SORUMLUSU: Dr. Tarık YILMAZ

BÖLÜM 1: SQL TEMELLERİ (DDL ve DML)

1.1. SQL Nedir?

SQL (Structured Query Language), ilişkisel veritabanlarını yönetmek ve tasarlamak için kullanılan standart bir dildir. Komutlar temel olarak iki ana kategoride incelenir:

- DDL (Data Definition Language):** Veri tanımlama dili. Tablo oluşturma, silme ve yapılandırma işlemlerini kapsar. (CREATE, ALTER, DROP)
- DML (Data Manipulation Language):** Veri işleme dili. Veri ekleme, silme, güncelleme işlemlerini kapsar. (INSERT, UPDATE, DELETE)

1.2. Tablo Oluşturma (CREATE TABLE)

Bir tablo oluşturulurken sütun isimleri, veri tipleri ve kısıtlayıcılar (constraints) belirlenir.

Temel Veri Tipleri:

- INT: Tamsayılar (Örn: ID, Stok).
- VARCHAR(n): Değişken uzunluklu metin (n: maksimum karakter).
- DECIMAL(p,s): Ondalıkli sayılar (Para birimi için idealdir).
- DATE: Tarih formatı (YYYY-AA-GG).

Örnek: Üyeler Tablosu Oluşturma

SQL

```
CREATE TABLE Uyeler (  
  UyeID INT PRIMARY KEY,          -- Birincil Anahtar  
  Ad VARCHAR(50) NOT NULL,        -- Boş geçilemez  
  Soyad VARCHAR(50) NOT NULL,  
  Eposta VARCHAR(100),  
  KayitTarihi DATE,  
  CezaPuanı DECIMAL(5,2) DEFAULT 0 -- Varsayılan değer 0  
);
```

1.3. Tablo Yapısını Değiştirme (ALTER) ve Silme (DROP)

SQL

-- Tabloyu tamamen silmek için:

DROP TABLE Uyeler;

-- Tabloya yeni sütun eklemek için:

ALTER TABLE Uyeler ADD TelefonNo VARCHAR(15);

1.4. Veri İşleme Komutları (DML)

Veri Ekleme (INSERT):

SQL

INSERT INTO Uyeler (UyeID, Ad, Soyad, KayitTarihi)

VALUES (1, 'Ahmet', 'Yılmaz', '2023-10-05');

Veri Güncelleme (UPDATE):

Dikkat: WHERE koşulu kullanılmazsa tüm kayıtlar güncellenir.

SQL

UPDATE Uyeler

SET Soyad = 'Yılmazer', CezaPuani = 10.50

WHERE UyeID = 1;

Veri Silme (DELETE):

Dikkat: WHERE koşulu kullanılmazsa tüm tablo içeriği silinir.

SQL

DELETE FROM Uyeler WHERE UyeID = 1;

1.5. Temel Sorgulama (SELECT)

Veritabanından veri çekmek için kullanılır.

SQL

-- Tüm sütunları getirme

SELECT * FROM Uyeler;

-- Filtreleme (WHERE) ve Sıralama (ORDER BY)

SELECT Ad, Soyad, CezaPuani

FROM Uyeler

WHERE CezaPuani > 5

ORDER BY CezaPuani DESC; -- Azalan sırada

BÖLÜM 2: İLİŞKİSEL SORGULAMA VE GRUPLAMA

2.1. Tablo Birleştirme (JOIN İşlemleri)

İlişkisel veritabanlarında veriler farklı tablolarda tutulur. Bu tabloları ortak sütunlar (Primary Key ve Foreign Key) üzerinden birleştirmek için JOIN kullanılır.

Senaryo: Musteriler ve Siparisler tabloları.

A) INNER JOIN (Kesişim):

Sadece her iki tabloda da eşleşen kayıtları getirir.

SQL

```
SELECT Musteriler.AdSoyad, Siparisler.Tutar
FROM Musteriler
INNER JOIN Siparisler ON Musteriler.MusteriID = Siparisler.MusteriID;
B) LEFT JOIN (Sol Birleşim):
```

Sol tablodaki (FROM) tüm kayıtları getirir. Sağ tabloda karşılığı yoksa NULL döner.

SQL

```
SELECT Musteriler.AdSoyad, Siparisler.SiparisID
FROM Musteriler
LEFT JOIN Siparisler ON Musteriler.MusteriID = Siparisler.MusteriID;
```

2.2. Agrege Fonksiyonlar ve Gruplama (GROUP BY)

Veriyi özetlemek ve istatistiksel sonuçlar üretmek için kullanılır.

Temel Fonksiyonlar:

- COUNT(*): Satır sayısı.
- SUM(): Toplam.
- AVG(): Ortalama.

GROUP BY Kullanımı:

Verileri belirli bir sütuna göre gruplayarak özetler.

SQL

```
SELECT Sehir, COUNT(*) as KisiSayisi
FROM Musteriler
GROUP BY Sehir;
```

2.3. Gruplanmış Veriyi Filtreleme (HAVING)

WHERE komutu ham veriyi filtrelerken, HAVING komutu gruplanmış (GROUP BY sonrası oluşan) veriyi filtreler.

Örnek: Toplam sipariş tutarı 10.000 TL'yi geçen müşteriler.

SQL

```
SELECT MusteriID, SUM(Tutar) as ToplamHarcama
FROM Siparisler
GROUP BY MusteriID
HAVING SUM(Tutar) > 10000;
```

BÖLÜM 3: İLERİ SORGULAMA TEKNİKLERİ

3.1. Alt Sorgular (Subqueries)

Bir sorgunun sonucunun, başka bir sorguda girdi olarak kullanılmasıdır.

Örnek: Ortalama fiyattan daha pahalı olan ürünler.

```
SQL
SELECT UrunAdi, Fiyat
FROM Urunler
WHERE Fiyat > (SELECT AVG(Fiyat) FROM Urunler);
```

Örnek: Liste döndüren alt sorgular (IN operatörü ile).

```
SQL
SELECT DersAdi FROM Dersler
WHERE DersID IN (
    SELECT DersID FROM OgrenciDersleri WHERE OgrenciID = 101
);
```

3.2. Küme Operatörleri (UNION)

İki farklı sorgunun sonucunu alt alta birleştirmek için kullanılır.

- UNION: Tekrar eden kayıtları eler.
- UNION ALL: Tekrarları elemeyen tüm sonuçları birleştirir.

```
SQL
SELECT Ad FROM EskiPersonel
UNION
SELECT Ad FROM YeniPersonel;
```

3.3. Görünümler (Views)

Karmaşık sorguları (örneğin uzun JOIN işlemlerini) sanal bir tablo olarak kaydetmeye yarar. Fiziksel olarak veri tutmaz, sadece sorguyu saklar.

```
SQL
-- View Oluşturma
CREATE VIEW OzetRapor AS
SELECT M.AdSoyad, S.Tarih, S.Tutar
FROM Musteriler M
INNER JOIN Siparisler S ON M.MusteriID = S.MusteriID;

-- View Kullanımı
SELECT * FROM OzetRapor WHERE Tutar > 500;
```

BÖLÜM 4: ER DİYAGRAMLARINDAN TABLOLARA DÖNÜŞÜM

Veritabanı tasarımında kavramsal modelin (ER Diyagramı) fiziksel modele (SQL Tabloları) dönüştürülmesi için temel kurallar şunlardır:

4.1. Varlıklar ve Nitelikler

Her varlık (Entity - Dikdörtgen) bir tabloya dönüşür. Nitelikler (Attribute - Elips) tablonun sütunları olur.

4.2. Bire-Çok (1:N) İlişkiler

İki tablo arasındaki ilişkide, "Çok" (N) olan taraftaki tabloya, "Bir" (1) olan tarafın birincil anahtarı **Yabancı Anahtar (Foreign Key)** olarak eklenir.

- **Örnek:** Fakülte (1) -> Bölüm (N)
- **İşlem:** Bölümler tablosuna FakulteID sütunu eklenir.

4.3. Çoktan-Çoğa (M:N) İlişkiler

İlişkisel veritabanlarında M:N ilişki fiziksel olarak kurulamaz. Bu ilişkiyi çözmek için **3. bir tablo (Ara Tablo / Junction Table)** oluşturulur.

- **Örnek:** Öğrenci <-> Ders
- **Çözüm:** DersKayit adında yeni bir tablo oluşturulur.
 - Bu tablo ÖğrenciID ve DersID sütunlarını içerir.
 - Her iki sütun da ilgili tablolara giden Yabancı Anahtarlardır.

SQL Örneği (Ara Tablo):

SQL

```
CREATE TABLE DersKayit (  
  KayitID INT PRIMARY KEY IDENTITY(1,1),  
  ÖğrenciID INT,  
  DersID INT,  
  Notu INT,  
  FOREIGN KEY (ÖğrenciID) REFERENCES Öğrenciler(ÖğrID),  
  FOREIGN KEY (DersID) REFERENCES Dersler(DersID)  
);
```